GoalAI Platform

AI-модуль для управления эффективностью персонала

Отчёт команды

Хакатон: «Внедрение ИИ в HR-процессы»
Март 2026

MVP WEBSITE: **https://goalhrtech.duckdns.org**
GITHUB: **https://github.com/ph1lomelaa/hr_tech.git**

Full name: Muslima Kosmagambetova
Gmail: muslima5671@gmail.com
tg: ph1lomelaa

1. Назначение системы

GoalAI Platform — AI-модуль, разработанный в рамках хакатона «Внедрение ИИ в HR-процессы». Система решает четыре прикладные задачи:

1. Оценка качества целей сотрудников по методологии SMART и смежным критериям.
2. Генерация стратегически связанных целей на основе ВНД, KPI подразделения и целей руководителя.
3. Контроль набора целей: количество, суммарный вес, дублирование, достижимость, стратегическая связка.
4. Агрегированная аналитика зрелости целеполагания по подразделениям и компании.

Решение реализовано как полнофункциональный веб-сервис с тремя ролевыми моделями (HR, руководитель, сотрудник), задеплоено на публичный стенд и доступно для демонстрации.

2. Функциональные возможности

2.1 Оценка качества целей

Система принимает текст цели сотрудника и выполняет оценку по пяти критериям SMART (Specific, Measurable, Achievable, Relevant, Time-bound). По каждому критерию выставляется балл от 0.0 до 1.0 с текстовым обоснованием. Рассчитывается агрегированный SMART-индекс. При выявлении слабых критериев система формирует конкретные рекомендации по улучшению и предлагает переформулированную версию цели.

Дополнительно определяются: тип цели (activity / output / impact), уровень стратегической связки (strategic / functional / operational) и конкретный источник связки — название документа ВНД, KPI или цель руководителя.

Реализована пакетная оценка: все цели сотрудника за квартал оцениваются одним запросом с формированием сводного индекса и перечня наиболее проблемных критериев.

2.2 Генерация целей

Система генерирует от 3 до 5 целей для конкретного сотрудника. Процесс генерации включает следующие этапы:

1. Извлечение релевантных фрагментов внутренних нормативных документов через гибридный RAG-поиск.
2. Загрузка актуальных KPI подразделения из таблицы kpi_timeseries.
3. Загрузка целей непосредственного руководителя для обеспечения каскадирования.
4. Формирование контекстуализированного промпта и генерация целей через LLM.
5. Автоматическая SMART-проверка каждой сгенерированной цели. При индексе ниже 0.7 цель переформулируется.

Каждая сгенерированная цель содержит привязку к конкретному документу ВНД: название документа, цитату из релевантного фрагмента, идентификатор секции и параграфа. Сотрудник может принять, отклонить или отредактировать предложенную цель через интерфейс.

2.3 Контроль набора целей

Система выполняет автоматический контроль набора целей сотрудника и генерирует предупреждения (алерты):

- Количество целей меньше минимального (3) или больше максимального (5) рекомендуемого значения.
- Суммарный вес целей отличается от 100%.
- Обнаружены дублирующие цели — проверка выполняется как лексически (SequenceMatcher + Jaccard), так и семантически (cosine similarity по эмбеддингам).
- Цель нереалистична по масштабу относительно исторических данных аналогичных ролей и подразделений.
- Цель классифицирована как activity-based; система предлагает переформулировку в результат-ориентированную.

2.4 Аналитика

Дашборд аналитики предоставляет данные по каждому подразделению и по компании в целом: средний SMART-индекс, распределение целей по типам, перечень наиболее слабых критериев, доля стратегически связанных целей, динамика показателей по кварталам. Рассчитывается интегральный индекс зрелости целеполагания подразделения с формированием рекомендаций для руководителя.

3. Архитектура решения

3.1 Контур выполнения

Система состоит из четырёх сервисов, связанных через Docker Compose:

```
Browser → Frontend (React + Vite, :8080) → API (FastAPI, :8000/:8002)
  ├─ PostgreSQL 17 (:5432)
  ├─ ChromaDB 0.5.18 (:8000 внутри docker-сети)
  └─ LLM API: Anthropic / Gemini / OpenAI
```

Фронтенд работает через относительный BASE = "/api/v1" с проксированием через Vite, что исключает жёсткую привязку к адресу бэкенда. Поддерживаются два режима запуска: полный стек (docker-compose.yml) и отдельный бэкенд для локальной разработки.

3.2 Технологический стек

Бэкенд

Слой	Технология	Версия
API	FastAPI	0.115.0
ASGI	Uvicorn	0.30.6
ORM	SQLAlchemy async	2.0.36
Драйвер PostgreSQL	asyncpg	0.30.0
Валидация	Pydantic	2.9.2
Settings	pydantic-settings	2.6.0
HTTP-клиент	httpx	0.27.2

AI / NLP

Слой	Технология	Примечание
Основной LLM	Anthropic	приоритет 1, если задан ANTHROPIC_API_KEY
Второй LLM	Gemini	приоритет 2, если задан API_KEY
Третий LLM	OpenAI	приоритет 3, если задан OPENAI_API_KEY
Локальные эмбеддинги	sentence-transformers	paraphrase-multilingual-MiniLM-L12-v2
Резерв эмбеддингов	Gemini embeddings	models/gemini-embedding-001
Векторная база	ChromaDB	гибридный RAG + лексический поиск по Postgres

Фронтенд

Слой	Технология
UI	React 18 + TypeScript
Сборка	Vite
Стилизация	Tailwind CSS
Компоненты	shadcn/ui
Данные	TanStack Query
Маршрутизация	React Router
Анимации	Framer Motion

3.3 Docker и развёртывание

Корневой docker-compose.yml поднимает четыре сервиса: backend-api (порт 8002), frontend (порт 8080), postgres (порт 5432), chromadb. Бэкенд слушает порт 8000 внутри контейнера и публикуется как 8002 на хосте. ChromaDB адресуется как chromadb:8000 внутри docker-сети. Фронтенд работает в режиме разработки с горячей перезагрузкой.

Дополнительно предусмотрен backend/docker-compose.yml для backend-ориентированной разработки (api на 8002, postgres на 5432, chromadb на 8001).

3.4 Конфигурация

Основные переменные окружения бэкенда (backend/app/config.py):

- DATABASE_URL, CHROMA_HOST, CHROMA_PORT, CHROMA_COLLECTION — подключение к базам данных.
- API_KEY (Gemini), OPENAI_API_KEY, ANTHROPIC_API_KEY — ключи LLM-провайдеров.
- EMBEDDING_MODEL — модель эмбеддингов (по умолчанию paraphrase-multilingual-MiniLM-L12-v2).
- AUTH_JWT_SECRET — секрет для подписи токенов.

Операционные AI-настройки: SMART_THRESHOLD, MIN_GOALS, MAX_GOALS, DUPLICATE_SIMILARITY_THRESHOLD, AI_*_MAX_CONCURRENCY, AI_*_RATE_LIMIT_PER_MINUTE, ANALYTICS_CACHE_TTL_SECONDS, LLM_CACHE_TTL_SECONDS, LLM_PROVIDER_COOLDOWN_SECONDS.

3.5 Структура бэкенда

```
backend/app/
├─ main.py, config.py, database.py, security.py, auth_jwt.py, ai_guardrails.py
├─ api/v1/  — auth, goals, evaluate, generate, employees, documents, analytics,
ai_logs
├─ services/  — smart_evaluator, goal_generator, analytics_service, rag_service,
│             goal_quality_rules, rule_based_evaluator
├─ ai/  — llm_client, embeddings, prompts
├─ vector_store/  — chroma_client, ingestion
├─ models/  — hr_models (схема дампа), ai_models (расширения)
├─ schemas/  — evaluation, generation, analytics
└─ utils/  — ai_trace, citation, document_scope, goal_fields
```

3.6 Жизненный цикл запуска бэкенда

Функция lifespan в main.py при старте приложения выполняет следующие действия:

1. Создание AI-таблиц расширения и performance-индексов (create_tables).
2. Проверка наличия записей в таблице document_chunks.
3. При отсутствии чанков — автоматический запуск ingestion документов в PostgreSQL и ChromaDB.
4. Использование advisory lock PostgreSQL для предотвращения двойного bootstrap при параллельных стартах.

Проект не требует отдельной инициализации через schema.sql. AI-структуры создаются поверх импортированного хакатонного дампа, что обеспечивает возможность интеграции с любой существующей HR-системой без миграции данных.

3.7 Модель авторизации и доступа

Система использует собственные подписанные JWT-like bearer-токены (auth_jwt.py). Реализованы три роли с гранулярными правилами доступа:

- hr — полный доступ ко всем данным и функциям системы.
- manager — доступ к собственным данным и данным прямых подчинённых.
- employee — доступ исключительно к собственным данным.

Дополнительные ограничения: аналитика по компании доступна только роли HR; аналитика по департаменту для менеджера ограничена его подразделением; согласование документов выполняется в два этапа — manager, затем hr.

На фронтенде реализован IdentitySwitcher для переключения роли и актёра. Данный компонент является частью демонстрационного сценария и обеспечивает возможность проверки системы от лица любого сотрудника.

3.8 AI-слой

Стратегия вызова LLM

Модуль llm_client.py реализует multi-provider стратегию с автоматическим переключением: Anthropic → Gemini → OpenAI. При недоступности всех провайдеров активируется правило-ориентированный fallback на уровне сервисов (rule_based_evaluator.py), обеспечивающий работоспособность системы без внешних API.

Механизмы надёжности

- Восстановление и очистка JSON-ответов от LLM.
- Удаление markdown-блоков из ответов.
- Устойчивый разбор JSON с обработкой типичных ошибок форматирования.
- TTL-кэш промптов в памяти для снижения нагрузки на LLM.
- Cooldown провайдера после повторяющихся ошибок.
- Trace-логирование всех AI-вызовов в logs/ai_trace.jsonl.

Эмбеддинги

Модуль embeddings.py реализует смешанную стратегию: основной путь — локальная модель sentence-transformers (paraphrase-multilingual-MiniLM-L12-v2); резервный путь — асинхронные Gemini embeddings API (models/gemini-embedding-001).

3.9 RAG-пайплайн

Извлечение релевантных документов реализовано через гибридный RAG-пайплайн:

1. Построение запроса в сервисном слое с генерацией 4–8 вариаций (по должности, подразделению, алиасам, фокус-направлению).
2. Параллельное векторное извлечение из ChromaDB.
3. Лексическое извлечение из PostgreSQL по таблице document_chunks (Full-Text Search с русскоязычным стеммером, fallback на ILIKE).
4. Объединение результатов через RRF (Reciprocal Rank Fusion, k=60).
5. Фильтрация по department_scope для обеспечения релевантности подразделению.
6. Обогащение метаданных: секция, параграф, номер чанка для точного цитирования.

Ingestion-пайплайн (vector_store/ingestion.py): загрузка активных документов → нормализация текста → чанкование (до 900 символов с перекрытием в 2 предложения) → генерация эмбеддингов → запись в ChromaDB → сохранение метаданных чанков в PostgreSQL.

3.10 Модель данных

Система использует два уровня данных: схему хакатонного дампа и AI-таблицы расширения, создаваемые приложением.

Основные модели HR-дампа

Таблица	Ключевые поля
departments	id, name, code, parent_id, is_active
positions	id, name, grade
employees	id, employee_code, full_name, email, department_id, position_id, manager_id, hire_date, is_active
documents	doc_id (uuid), doc_type, title, content, valid_from/to, owner_department_id, department_scope (jsonb), keywords (text[])
goals	goal_id (uuid), employee_id, department_id, goal_text, year, quarter, metric, deadline, weight, status, priority
goal_events	id, goal_id, event_type, actor_id, old_status, new_status, old_text, new_text, metadata
goal_reviews	id, goal_id, reviewer_id, verdict, comment_text
kpi_catalog	metric_key (PK), name, unit, description
kpi_timeseries	metric_key, period_date, value_num, scope_type + связи с департаментом/сотрудником/проектом

AI-таблицы расширения (ai_models.py)

Таблица	Назначение
smart_evaluations	Результаты SMART-оценок целей
generation_sessions	Сессии генерации целей
suggested_goals	Предложенные AI цели-кандидаты
goal_sources	Привязка целей к документам ВНД (источник + цитата)
document_reviews	История согласований документов
document_chunks	Чанки документов для RAG-поиска
department_maturity_cache	Кэш индекса зрелости целеполагания
goal_alerts	Предупреждения по набору целей сотрудника

Архитектура намеренно не является greenfield-решением: AI-слой адаптируется к схеме дампа организаторов, а не заменяет её собственной схемой. Это обеспечивает возможность интеграции модуля в существующие HR-системы.

3.11 Поверхность API

Базовый префикс: /api/v1

Группа	Эндпоинты

Авторизация	POST /auth/impersonate, GET /auth/options, GET /auth/whoami
Цели	GET/POST /goals, GET /goals/{id}, PATCH /goals/{id}/status, GET /goals/{id}/events, DELETE /goals/{id}
Оценка	POST /evaluate/goal, POST /evaluate/goal/{id}, POST /evaluate/batch, POST /evaluate/backfill
Генерация	POST /generate/goals, POST /generate/accept, POST /generate/rewrite, POST /generate/reject
Сотрудники	GET /employees, GET /{id}, GET /{id}/goals, GET /{id}/manager-goals, GET /{id}/alerts, PATCH /{id}/alerts/{alert_id}/read
Документы	GET /documents, GET /documents/search, GET /{doc_id}, GET/POST /{doc_id}/approvals
Аналитика	GET /analytics/department/{id}, GET /analytics/company, POST /analytics/refresh
AI-логи	GET /ai/logs

3.12 Архитектура фронтенда

Интерфейс построен на DashboardLayout.tsx с адаптивным сайдбаром и верхней панелью, зависящими от роли. Реализованы переключение темы и компонент IdentitySwitcher.

Маршруты

Роль	Маршруты
Публичное	/ → LandingPage
HR	/hr, /hr/goals, /hr/goals/:goalId, /hr/generate, /hr/documents, /hr/employees, /hr/analytics
Manager	/manager, /manager/team-goals, /manager/team-goals/:goalId, /manager/my-goals, /manager/generate, /manager/documents, /manager/feedback, /manager/employees
Employee	/employee, /employee/goals, /employee/goals/:goalId, /employee/generate, /employee/documents, /employee/feedback

API-клиент (lib/api.ts) использует относительный BASE = "/api/v1", bearer-токен через /auth/impersonate. Состояние выбранного сотрудника хранится раздельно по ролям в localStorage. Кэширование и оркестрация данных — через React Query.

3.13 Основные бизнес-потоки

SMART-оценка

1. Пользователь отправляет текст цели или выбирает существующую из БД.
2. Бэкенд обогащает контекст данными сотрудника, департамента и RAG-контекстом.
3. Сервис оценки вызывает LLM. При недоступности провайдера активируется rule-based fallback.
4. Результат сохраняется в smart_evaluations и возвращается клиенту.

Генерация целей

1. Выбирается целевой сотрудник.
2. Загружаются цели руководителя для каскадирования.
3. Извлекаются релевантные документы через гибридный RAG.
4. LLM генерирует 3–5 кандидатных целей.
5. Каждая цель проходит SMART-проверку и дополнительные валидации.
6. Предложения сохраняются в suggested_goals.
7. Пользователь принимает или отклоняет. Принятая цель становится записью в goals с привязкой goal_source.

Аналитика

1. Сервис агрегирует оценённые цели по подразделениям.
2. Рассчитывает индекс зрелости, слабые критерии, распределение типов.
3. Результат кэшируется в department_maturity_cache.
4. Принудительный пересчёт — через refresh=true или POST /analytics/refresh.

3.14 Производительность и guardrails

При старте создаются performance-индексы: по сотруднику и периоду (goals), по департаменту и периоду, по статусу прочтения (alerts), FTS-индекс для document_chunks.

Модуль ai_guardrails.py обеспечивает: rate limits по типу операции (generate, evaluate, batch), семафоры для ограничения параллелизма, тайм-ауты ожидания в очереди.

Устойчивость LLM-слоя: кэш промптов, cooldown провайдеров после сбоев, восстановление JSON перед fallback, rule-based fallback.

1. Переключить роль на менеджера. Продемонстрировать каскадирование целей к подчинённым.
2. Показать переформулировку слабой цели: исходная формулировка → улучшенная версия.